LAB REPORT 6

Introduction to Mininet-WiFi: Stations & Access Points

Student Name	Hadis Rodpradit
Student ID	6630613039
Program	Digital Engineering International Program, College of Computing, PSU Phuket
Course	977-337 Network Programming and Simulation
Week	6
Environment	Ubuntu Linux VM, Mininet-WiFi Python API (`mn_wifi`), mac80211_hwsim virtual radios
Source file	`lab06_wifi_intro/wifi_topo.py` in the netprog-2026 repository

1. Objective

This lab moves from wired Mininet to Mininet-WiFi. I built a topology with one access point `ap1` (SSID `netprog-ap`, 802.11g, channel 1, range 15 m) and wireless stations using `addStation()`, `addAccessPoint()` and `configureWifiNodes()`, checked association with `iw dev <sta>-wlan0 link`, measured throughput through the AP, showed that a station outside the AP's range cannot associate, and captured real ICMP frames on a station's wireless interface.

2. Final Script (wifi_topo.py)

There is no `addLink()` between a station and the AP — association happens automatically from each station's position relative to the AP's range. Positions are in meters, so the distance from ap1 (50,50,0) is $|x - 50|$ for every station in this lab:

Node	Position	Distance to ap1	Inside range=15?
sta1	45,50,0	5 m	Yes
sta2	55,50,0	5 m	Yes
sta3 (before)	90,50,0	**40 m**	**No**
sta3 (after)	60,50,0	10 m	Yes

```
from mn_wifi.net import Mininet_wifi
from mn_wifi.cli import CLI
from mininet.log import setLogLevel, info

def topology():
    net = Mininet_wifi()

    info("*** Creating nodes\n")
    sta1 = net.addStation('sta1', position='45,50,0')
    sta2 = net.addStation('sta2', position='55,50,0')
    sta3 = net.addStation('sta3', position='60,50,0')   # moved into range (was '90,50,0')
    ap1 = net.addAccessPoint('ap1', ssid='netprog-ap', mode='g', channel='1',
                             position='50,50,0', range=15)

    info("*** Configuring wifi nodes\n")
    net.configureWifiNodes()

    info("*** Starting network\n")
    net.build()
    net.start()

    net.pingAll()
    CLI(net)

    net.stop()

if __name__ == "__main__":
    setLogLevel("info")
    topology()
```

3. Lab Tasks and Results

Task 1 — Minimal AP + Two Stations (Part B)

```
mininet-wifi> pingall
*** Ping: testing ping reachability
sta1 -> *** sta1 : ('ping -c1  10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.845 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.845/0.845/0.845/0.000 ms
sta2
sta2 -> *** sta2 : ('ping -c1  10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.757 ms

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.757/0.757/0.757/0.000 ms
sta1
*** Results: 0% dropped (2/2 received)
```

Figure 1: Minimal topology (ap1, sta1, sta2): pingall reports 0% dropped (2/2 received)

With only sta1 and sta2 (each 5 m from ap1), both stations associated and reached each other through the AP: **0% dropped (2/2 received)**, with RTTs of 0.845 ms (sta1 → sta2) and 0.757 ms (sta2 → sta1). This Mininet-WiFi build prints each underlying `ping -c1` command, which shows that pingall is simply running the real ping program inside each station's namespace.

Task 2 — Association and Signal Information (Part C)

```
mininet-wifi> net
sta1 sta1-wlan0:wifi
sta2 sta2-wlan0:wifi
ap1 ap1-wlan1:wifi ap1-wlan1:wifi
mininet-wifi>
mininet-wifi> sta1 iw dev sta1-wlan0 link
Connected to 02:00:00:00:02:00 (on sta1-wlan0)
        SSID: netprog-ap
        freq: 2412.0
        RX: 60839 bytes (1319 packets)
        TX: 1776 bytes (19 packets)
        signal: -49 dBm
        rx bitrate: 54.0 MBit/s
        tx bitrate: 1.0 MBit/s
        bss flags: short-slot-time
        dtim period: 2
        beacon int: 100
mininet-wifi>
mininet-wifi> sta2 iw dev sta2-wlan0 link
Connected to 02:00:00:00:02:00 (on sta2-wlan0)
        SSID: netprog-ap
        freq: 2412.0
        RX: 68065 bytes (1478 packets)
        TX: 1864 bytes (20 packets)
        signal: -49 dBm
        rx bitrate: 54.0 MBit/s
        tx bitrate: 48.0 MBit/s
        bss flags: short-slot-time
        dtim period: 2
        beacon int: 100
```

Figure 2: net output and iw dev sta1-wlan0 link / sta2-wlan0 link

Field	sta1-wlan0	sta2-wlan0	Meaning
Connected to	02:00:00:00:02:00	02:00:00:00:02:00	BSSID = MAC of ap1's radio
SSID	**netprog-ap**	**netprog-ap**	the SSID set in addAccessPoint()
freq	2412.0 MHz	2412.0 MHz	802.11 channel 1 (2.4 GHz)
signal	**−49 dBm**	**−49 dBm**	strong signal (5 m from AP)
rx / tx bitrate	54.0 / 1.0 Mbit/s	54.0 / 48.0 Mbit/s	802.11g rates (max 54 Mbit/s)
beacon int / dtim	100 / 2	100 / 2	AP beacon every 100 TU (≈102 ms)

Both stations are associated with the same BSSID and SSID `netprog-ap` on 2412 MHz, which matches `mode='g', channel='1'`. The rx bitrate of 54 Mbit/s is the maximum 802.11g rate. sta1's tx bitrate of only 1.0 Mbit/s is the rate-control algorithm starting at the lowest rate because sta1 had sent very little (19 packets), while sta2 had already adapted up to 48 Mbit/s. The `net` output also shows the difference from wired Mininet: interfaces are `wlan0:wifi` with no peer, instead of `eth0:s1-eth1` pairs.

Task 3 — Bandwidth Through the AP (Part D)

```
mininet-wifi> iperf sta1 sta2
*** Iperf: testing TCP bandwidth between sta1 and sta2
*** Results: ['10.0 Mbits/sec', '10.0 Mbits/sec']
```

Figure 3: iperf sta1 sta2 through ap1

The TCP throughput from sta1 to sta2 through ap1 was **10.0 Mbits/sec** in both directions. This is very different from the 30 Gbit/s measured between wired hosts in Lab 5.1: traffic now crosses the emulated wireless path (sta1 → ap1 → sta2), and the identical, round 10.0 Mbit/s figure indicates a rate limit that Mininet-WiFi applies to the stations' wireless interfaces with Linux traffic control, rather than the CPU-bound speed of a plain veth pair. It is also realistically below the 54 Mbit/s 802.11g PHY rate, since real 802.11g delivers only about 20–25 Mbit/s of TCP goodput and every frame through an AP uses the shared channel twice.

Task 4 — Out-of-Range Station, Before Moving (Part E)

```
mininet-wifi> pingall
*** Ping: testing ping reachability
sta1 -> *** sta1 : ('ping -c1  10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.829 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.829/0.829/0.829/0.000 ms
sta2 *** sta1 : ('ping -c1  10.0.0.3',)
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
From 10.0.0.1 icmp_seq=1 Destination Host Unreachable

--- 10.0.0.3 ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

X
sta2 -> *** sta2 : ('ping -c1  10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.855 ms

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.855/0.855/0.855/0.000 ms
sta1 *** sta2 : ('ping -c1  10.0.0.3',)
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
From 10.0.0.2 icmp_seq=1 Destination Host Unreachable

--- 10.0.0.3 ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

X
sta3 -> *** sta3 : ('ping -c1  10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=1 Destination Host Unreachable

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

X *** sta3 : ('ping -c1  10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
From 10.0.0.3 icmp_seq=1 Destination Host Unreachable

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 0 received, +1 errors, 100% packet loss, time 0ms

X
*** Results: 66% dropped (2/6 received)
mininet-wifi>
mininet-wifi>
mininet-wifi> sta3 iw dev sta3-wlan0 link
Not connected.
```

Figure 4: sta3 at 90,50,0 (40 m from ap1): pingall 66% dropped and iw reports Not connected

With sta3 at `90,50,0` — 40 m from ap1, far beyond `range=15` — every ping to or from sta3 failed with **Destination Host Unreachable**, while sta1 ↔ sta2 still worked: **66% dropped (2/6 received)**. `sta3 iw dev sta3-wlan0 link` prints **Not connected.**, so sta3 never associated with ap1. The error is "Host Unreachable" (ARP got no answer) and not a timeout, because without association no frame from sta3 can reach the AP at all.

Task 5 — Station Moved Into Range, After Moving (Part E)

```
mininet-wifi> pingall
*** Ping: testing ping reachability
sta1 -> *** sta1 : ('ping -c1  10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.831 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.831/0.831/0.831/0.000 ms
sta2 *** sta1 : ('ping -c1  10.0.0.3',)
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=1.04 ms

--- 10.0.0.3 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 1.042/1.042/1.042/0.000 ms
sta3
sta2 -> *** sta2 : ('ping -c1  10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.744 ms

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.744/0.744/0.744/0.000 ms
sta1 *** sta2 : ('ping -c1  10.0.0.3',)
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=0.976 ms

--- 10.0.0.3 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.976/0.976/0.976/0.000 ms
sta3
sta3 -> *** sta3 : ('ping -c1  10.0.0.1',)
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.998 ms

--- 10.0.0.1 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.998/0.998/0.998/0.000 ms
sta1 *** sta3 : ('ping -c1  10.0.0.2',)
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.979 ms

--- 10.0.0.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.979/0.979/0.979/0.000 ms
sta2
*** Results: 0% dropped (6/6 received)
mininet-wifi>
mininet-wifi> sta3 iw dev sta3-wlan0 link
Connected to 02:00:00:00:03:00 (on sta3-wlan0)
        SSID: netprog-ap
        freq: 2412.0
        RX: 13021 bytes (245 packets)
        TX: 2160 bytes (22 packets)
        signal: -49 dBm
        rx bitrate: 54.0 MBit/s
        tx bitrate: 36.0 MBit/s
        bss flags: short-slot-time
        dtim period: 2
        beacon int: 100
```

Figure 5: sta3 at 60,50,0 (10 m from ap1): pingall 0% dropped and iw shows association with netprog-ap

After changing only sta3's position to `60,50,0` (10 m from ap1, inside the 15 m range), sta3 associated with ap1 (`Connected to 02:00:00:00:03:00`, SSID `netprog-ap`, signal −49 dBm, freq 2412 MHz) and pingall reported **0% dropped (6/6 received)**. The only change between Tasks 4 and 5 was one coordinate, which shows that association is decided purely by geometry (distance vs. range) in this model.

Note: the BSSID shown here (…:03:00) differs from the earlier run (…:02:00) because Mininet-WiFi assigns MAC addresses by node creation order, and adding sta3 shifted ap1 by one. The signal is −49 dBm at both 5 m

and 10 m because, without wmediumd's propagation model, the default model reports a fixed signal for any station inside the range.

Part F — Capturing Real Frames on a Wireless Interface

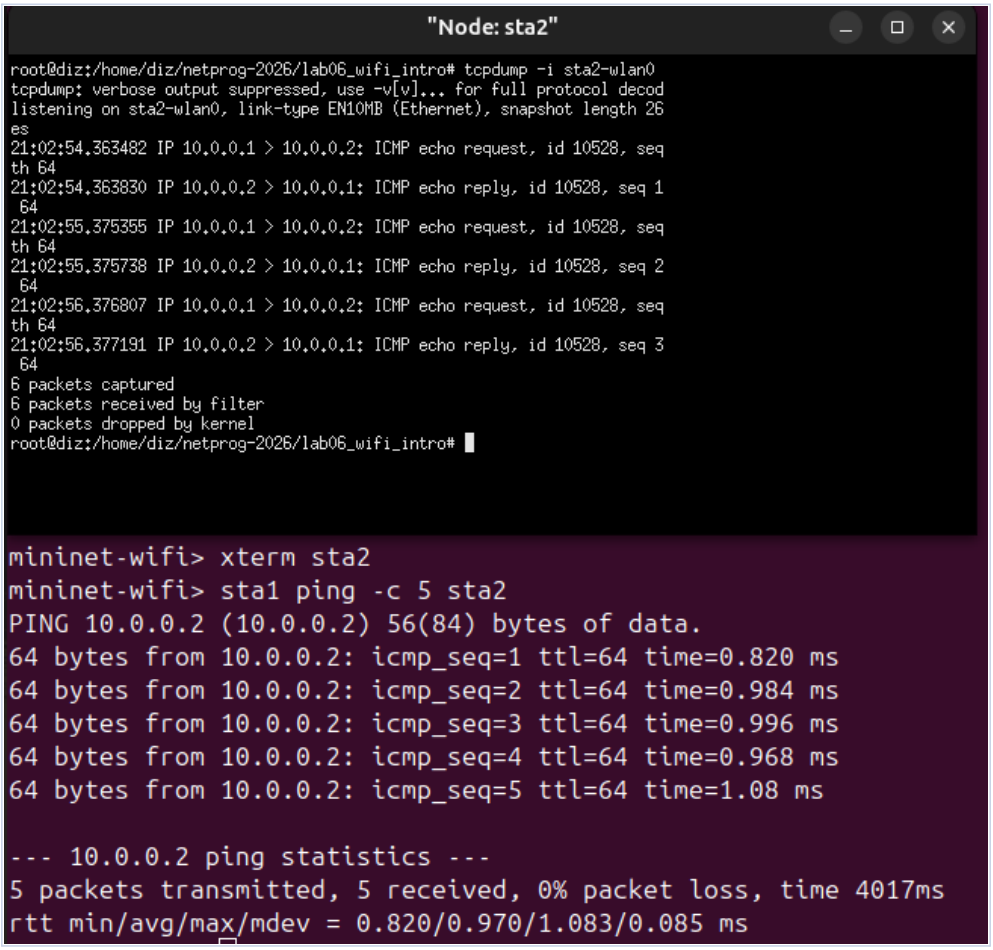


Figure 6: tcpdump -i sta2-wlan0 -c 6 in xterm "Node: sta2" while sta1 pings sta2

Because a background `tcpdump ... &` does not print its output to the Mininet-WiFi CLI, the capture was run in an xterm on sta2 (`xterm sta2`), the same method as in Lab 5.1. While `sta1 ping -c 5 sta2` ran (5/5 received, RTT avg 0.970 ms), tcpdump on `sta2-wlan0` captured **6 packets**: three **ICMP echo requests** (10.0.0.1 → 10.0.0.2) and three **ICMP echo replies** (10.0.0.2 → 10.0.0.1), id 10528, seq 1–3, with 0 packets dropped by the kernel. No ARP frames appear because sta1 and sta2 had already learned each other's MAC addresses from the earlier pingall.

These are real packets produced by the real Linux network stack and the real `ping` program, but they reached sta2 over a simulated 802.11 association through ap1 instead of a simulated Ethernet cable — the same emulation principle as Lab 5.1 Part F, now on a wireless data path. tcpdump reports link-type EN10MB (Ethernet) because a Wi-Fi interface in managed mode hands frames to the OS with an Ethernet-style header after the 802.11 layer has processed them.

Task 6 — Mininet vs. Mininet-WiFi

Switch vs. access point: a Mininet switch is an Open vSwitch bridge whose ports are wired veth pairs created with `addLink()`, whereas a Mininet-WiFi AP is a switch with a virtual radio (`ap1-wlan1`) running hostapd, which broadcasts an SSID and lets stations join it with no `addLink()` at all — membership depends on position and range, as Tasks 4–5 showed. **Host vs. station:** a host has a wired `eth0` permanently connected to a switch port, while a station has a `wlan0` interface that must *associate* first; sta3 at 40 m existed and had an IP address but was `Not connected.` and unreachable, something that cannot happen to a wired Mininet host.

4. Verification Checklist

#	Checkpoint	Evidence	✓
1	Minimal AP + 2-station topology (Part B) runs and pingall reports 0% dropped	Figure 1	✓
2	iw dev sta1-wlan0 link (Part C) shows a successful association with the correct SSID	Figure 2	✓
3	iperf sta1 sta2 (Part D) reports a nonzero bandwidth figure (10.0 Mbit/s)	Figure 3	✓
4	Out-of-range sta3 (Part E, before moving) fails to associate and fails pingall	Figure 4	✓
5	In-range sta3 (Part E, after moving) associates successfully and passes pingall	Figure 5	✓
6	tcpdump capture (Part F) shows real ICMP frames on a station's wlan interface	Figure 6	✓

5. Reflection Questions

Q1. Compare a Mininet switch and a Mininet-WiFi access point: what does each emulate at the OS level, and why can't a plain Mininet switch provide wireless connectivity?

A Mininet switch is an Open vSwitch (or Linux) bridge in the kernel whose ports are veth pairs — virtual Ethernet cables with a fixed peer at each end. A Mininet-WiFi AP is also an OVS bridge, but one of its ports is a virtual 802.11 radio created by the `mac80211_hwsim` kernel module, with hostapd running on it to send beacons, advertise the SSID and handle authentication and association. A plain switch has no radio interface, no 802.11 MAC layer and no hostapd, so there is nothing a station's `wlan0` could scan for or associate with; it can only forward frames between cables that `addLink()` has already connected.

Q2. What does net.configureWifiNodes() do, and why must it be called after all addStation()/addAccessPoint() calls but before net.build()?

`configureWifiNodes()` turns the declared nodes into real wireless devices: it loads `mac80211_hwsim` with one virtual radio for every station and AP, moves each `wlanX` into its node's namespace, sets mode/channel/SSID and starts hostapd on the AP, and applies the positions and ranges used by the propagation model. It must come after every `addStation()`/`addAccessPoint()` because it needs the complete list of nodes to know how many radios to create — a node added afterwards would have no radio. It must come before `net.build()` because building and starting the network brings these interfaces up and lets stations associate; if the wireless interfaces were not configured yet, stations would never associate and pingall would show 100% dropped.

Q3. Why did the out-of-range station fail to associate, and what real-world phenomenon does range approximate?

sta3 at (90,50,0) was 40 m from ap1 but the AP's `range` was only 15 m. Mininet-WiFi's default model only lets a station hear the AP and associate when the straight-line distance is within range, so sta3 never received a usable beacon or probe response, stayed `Not connected.`, and every ping returned Destination Host Unreachable. The range parameter approximates **path loss / signal attenuation**: in real Wi-Fi the received signal power falls with distance (and through walls and obstacles) until it drops below the receiver's sensitivity threshold, giving each AP a limited coverage area beyond which a device cannot connect.

6. Conclusion

The Mininet-WiFi topology worked as expected: sta1 and sta2 associated with `netprog-ap` on 2412 MHz at −49 dBm and reached each other with 0% loss, and TCP throughput through the AP was 10.0 Mbit/s. A station 40 m from an AP with a 15 m range could not associate and was unreachable, and moving it to 10 m fixed both association and pingall with no other change, which shows that Wi-Fi membership in Mininet-WiFi depends on position and range, not on explicit links.